

Regression Interpretation, Classification Metrics & Advertising Prediction

Cameron Batts — Linear Regression · Logistic Regression · Classification Metrics

```
In [20]: import joblib

import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.linear_model import LinearRegression
```

```
In [21]: multiple_linear_regression = joblib.load('linear_regression.joblib')
```

```
In [22]: multiple_linear_regression.summary()
```

Out[22]:

OLS Regression Results							
Dep. Variable:	log_selling_price		R-squared:	0.779			
Model:	OLS		Adj. R-squared:	0.777			
Method:	Least Squares		F-statistic:	583.4			
Date:	Mon, 11 May 2026		Prob (F-statistic):	0.00			
Time:	12:02:22		Log-Likelihood:	-2125.1			
No. Observations:	4340		AIC:	4304.			
Df Residuals:	4313		BIC:	4476.			
Df Model:	26						
Covariance Type:	nonrobust						
		coef	std err	t	P> t	[0.025	0.975]
	const	12.4959	0.059	212.070	0.000	12.380	12.611
	year	0.1178	0.002	66.227	0.000	0.114	0.121
	km_driven	-0.0008	0.000	-5.354	0.000	-0.001	-0.001
	fuel_Diesel	0.3954	0.015	27.002	0.000	0.367	0.424
	fuel_Other	-0.1087	0.051	-2.148	0.032	-0.208	-0.009
	seller_type_Individual	-0.1047	0.016	-6.728	0.000	-0.135	-0.074
	seller_type_Trustmark	0.3715	0.042	8.849	0.000	0.289	0.454

Dealer						
owner_Other	-0.0820	0.042	-1.963	0.050	-0.164	-0.000
owner_Second Owner	-0.0328	0.015	-2.119	0.034	-0.063	-0.002
owner_Third Owner	-0.0979	0.026	-3.798	0.000	-0.148	-0.047
brand_BMW	0.1236	0.082	1.513	0.130	-0.037	0.284
brand_Chevrolet	-1.7207	0.060	-28.876	0.000	-1.838	-1.604
brand_Datsun	-1.7509	0.084	-20.753	0.000	-1.916	-1.585
brand_Fiat	-1.6262	0.083	-19.481	0.000	-1.790	-1.463
brand_Ford	-1.3177	0.058	-22.858	0.000	-1.431	-1.205
brand_Honda	-1.1191	0.058	-19.433	0.000	-1.232	-1.006
brand_Hyundai	-1.3317	0.054	-24.594	0.000	-1.438	-1.226
brand_Mahindra	-1.1678	0.056	-20.824	0.000	-1.278	-1.058
brand_Maruti	-1.4619	0.054	-27.191	0.000	-1.567	-1.357
brand_Mercedes-Benz	0.2986	0.084	3.540	0.000	0.133	0.464
brand_Nissan	-1.3616	0.072	-18.971	0.000	-1.502	-1.221
brand_Other	-0.2968	0.084	-3.536	0.000	-0.461	-0.132
brand_Renault	-1.5154	0.062	-24.615	0.000	-1.636	-1.395
brand_Skoda	-1.0981	0.070	-15.577	0.000	-1.236	-0.960
brand_Tata	-1.8146	0.056	-32.224	0.000	-1.925	-1.704
brand_Toyota	-0.7292	0.059	-12.350	0.000	-0.845	-0.613
brand_Volkswagen	-1.2389	0.064	-19.276	0.000	-1.365	-1.113

Omnibus:	133.156	Durbin-Watson:	1.874
Prob(Omnibus):	0.000	Jarque-Bera (JB):	326.637
Skew:	0.109	Prob(JB):	1.18e-71
Kurtosis:	4.326	Cond. No.	3.04e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.04e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Holding all other variables constant, a car branded as a Maruti is associated with an approximate 76.82% decrease in selling price compared to the baseline brand. Using $(e^{-1.4619})-1$

Holding all other variables such as mileage and fuel type constant, each one-year increase in the car's model year is associated with an approximate 12.5% increase in the selling price using $(e^{0.1178})-1=12\%$

Now, consider again the logistic regression model that we used to model the probability that a customer subscribes a term deposit for the Bank Marketing dataset.

```
In [23]: logistic_regression_model = joblib.load('logistic_regression.joblib')
```

Here is the model summary:

```
In [24]: logistic_regression_model.summary()
```

```
Out [24]:
```

Logit Regression Results							
Dep. Variable:	y	No. Observations:	45211				
Model:	Logit	Df Residuals:	45185				
Method:	MLE	Df Model:	25				
Date:	Mon, 11 May 2026	Pseudo R-squ.:	0.09451				
Time:	12:02:22	Log-Likelihood:	-14774.				
converged:	True	LL-Null:	-16315.				
Covariance Type:	nonrobust	LLR p-value:	0.000				
	coef	std err	z	P> z	[0.025	0.975]	
const	-1.2729	0.103	-12.347	0.000	-1.475	-1.071	
age	0.0045	0.002	2.407	0.016	0.001	0.008	
balance	0.0211	0.004	5.145	0.000	0.013	0.029	
campaign	-0.1321	0.009	-15.418	0.000	-0.149	-0.115	
previous	0.0902	0.006	14.719	0.000	0.078	0.102	
job_blue-collar	-0.2321	0.062	-3.745	0.000	-0.354	-0.111	
job_entrepreneur	-0.4052	0.108	-3.759	0.000	-0.616	-0.194	
job_housemaid	-0.3983	0.114	-3.490	0.000	-0.622	-0.175	

job_management	-0.1714	0.062	-2.773	0.006	-0.293	-0.050
job_retired	0.4701	0.080	5.876	0.000	0.313	0.627
job_self-employed	-0.2233	0.094	-2.381	0.017	-0.407	-0.039
job_services	-0.1902	0.071	-2.673	0.008	-0.330	-0.051
job_student	0.4715	0.092	5.126	0.000	0.291	0.652
job_technician	-0.2076	0.058	-3.563	0.000	-0.322	-0.093
job_unemployed	0.0513	0.091	0.562	0.574	-0.128	0.230
job_unknown	-0.2314	0.196	-1.182	0.237	-0.615	0.152
marital_divorced	-0.1844	0.056	-3.282	0.001	-0.294	-0.074
marital_married	-0.3437	0.039	-8.907	0.000	-0.419	-0.268
education_secondary	0.1726	0.054	3.173	0.002	0.066	0.279
education_tertiary	0.4020	0.063	6.388	0.000	0.279	0.525
education_unknown	0.2823	0.087	3.234	0.001	0.111	0.453
default_yes	-0.3472	0.148	-2.352	0.019	-0.637	-0.058
housing_yes	-0.6474	0.033	-19.509	0.000	-0.712	-0.582
loan_yes	-0.5841	0.051	-11.408	0.000	-0.684	-0.484
contact_telephone	-0.2077	0.061	-3.415	0.001	-0.327	-0.089
contact_unknown	-1.1324	0.049	-23.143	0.000	-1.228	-1.036

And these are the model's coefficients after exponentiation.

```
In [25]: list(zip(logistic_regression_model.params.index, np.exp(logistic_regre
```

```
Out[25]: [('const', np.float64(0.28003051493142306)),
('age', np.float64(1.0044752145380202)),
('balance', np.float64(1.0213064724474892)),
('campaign', np.float64(0.876277499096245)),
('previous', np.float64(1.0944009330533633)),
('job_blue-collar', np.float64(0.7928585082022298)),
('job_entrepreneur', np.float64(0.6668533914136381)),
('job_housemaid', np.float64(0.6714553977781065)),
('job_management', np.float64(0.8424780508168234)),
('job_retired', np.float64(1.6002173552302916)),
('job_self-employed', np.float64(0.7998729526865037)),
('job_services', np.float64(0.826807369872747)),
('job_student', np.float64(1.6023642798054696)),
('job_technician', np.float64(0.8125498927833215)),
('job_unemployed', np.float64(1.0526698376921388)),
('job_unknown', np.float64(0.7933912487973834)),
('marital_divorced', np.float64(0.8316200897732506)),
('marital_married', np.float64(0.7091613334472329)),
('education_secondary', np.float64(1.1884203914695166)),
('education_tertiary', np.float64(1.494813313611824)),
('education_unknown', np.float64(1.3262240064333748)),
('default_yes', np.float64(0.7066392112323853)),
('housing_yes', np.float64(0.5233851851737711)),
('loan_yes', np.float64(0.5576202171093175)),
('contact_telephone', np.float64(0.8124475352065605)),
('contact_unknown', np.float64(0.32225851730263305))]
```

Holding all other variables constant, being self-employed is associated with a 20% decrease (less than 1), in the odds of subscribing to a term deposit compared to the baseline job category. $0.7998 - 1 = -0.2002$

Holding all other factors constant, for every one-unit increase in the customer's balance, the odds of subscribing to a term deposit increase by approximately 2.13%. Because this value is greater than 1, it indicates a positive relationship. $1.0213 - 1 = 0.0213$ (or 2.13%).

Consider the following confusion matrix:

	Predictions	0	1
True labels	0	100	20
	1	120	80

True Negatives (TN): 100 (Correctly predicted "No")

False Positives (FP): 20 (Incorrectly predicted "Yes")

False Negatives (FN): 120 (Incorrectly predicted "No")

True Positives (TP): 80 (Correctly predicted "Yes")

Total Observations: $100+20+120+80=320$

Accuracy(Overall correctness)= $TP+TN/Total = 180/320 = 0.563$ or 56.3%

Precision (Reliability of a "Yes" prediction)= $TP/TP+FP = 80/100 = 0.8$ or 80%

Recall (Ability to find all actual "Yes")= $TP/TP+FN = 80/(80+120) = 0.4$ or 40%

Specificity (Ability to find all actual "No")= $TN/TN+FP = 100/(100+20) = 0.833$ or 83.3%

Now, let's consider the following marketing data.

```
In [26]: marketing_data = pd.read_csv('marketing.csv')
```

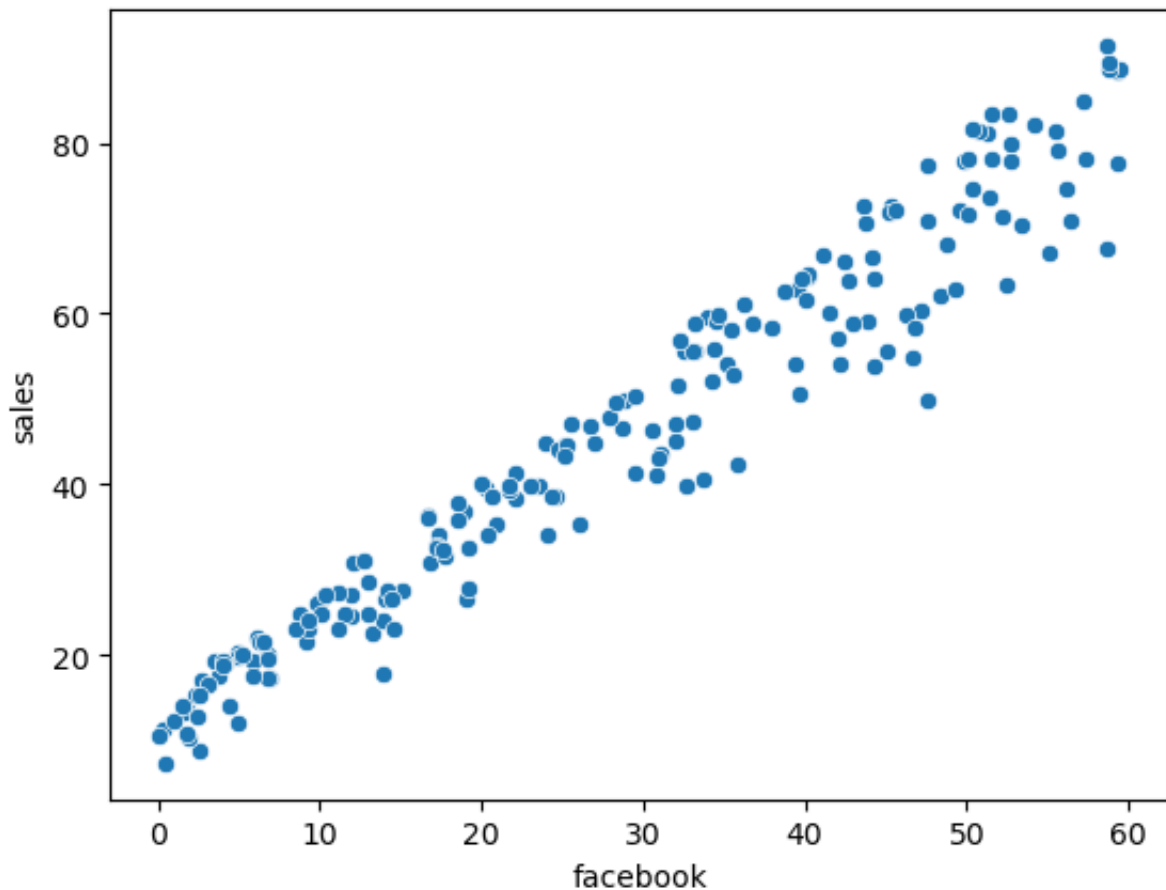
```
In [27]: marketing_data.head()
```

```
Out[27]:
```

	facebook	sales
0	45.36	72.52
1	47.16	60.48
2	55.08	67.16
3	49.56	72.20
4	12.96	28.48

- **facebook** measures the budget (in 1000s of USD) spent on an online marketing campaign for a product
- **sales** measures the sales (in 1000s of USD) induced by the campaign.

```
In [28]: _ = sns.scatterplot(x='facebook', y='sales', data=marketing_data)
```



```
In [29]: X = marketing_data['facebook'].values.reshape(-1, 1)
Y = marketing_data['sales']

Simple_Linear_Regression = LinearRegression()
Simple_Linear_Regression = LinearRegression().fit(X, Y)
Simple_Linear_Regression.score(X, Y)
```

Out[29]: 0.9463557108459727

R2 approximately 94.6% of the variation in sales can be explained by the Facebook marketing budget

```
In [30]: prediction_data = pd.DataFrame({'facebook': pd.Series(50)})

prediction = Simple_Linear_Regression.predict(prediction_data.values)
print(f"The predicted sales for a $50,000 budget is: {prediction[0]:.2f}")
```

The predicted sales for a \$50,000 budget is: 71.83

Formula used by the summary: $71.83 = \text{Intercept_coef} + (\text{facebook_coef} \times 50)$

```
In [31]: import statsmodels.formula.api as smf
simple_linear_regression_sm = smf.ols('sales ~ facebook', data=marketi
```

```
In [32]: print(simple_linear_regression_sm.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          sales    R-squared:
0.946
Model:                  OLS      Adj. R-squared:
0.946
Method:                 Least Squares    F-statistic:
3493.
Date:                   Mon, 11 May 2026    Prob (F-statistic):          9.
72e-128
Time:                   12:02:24    Log-Likelihood:
-609.40
No. Observations:      200    AIC:
1223.
Df Residuals:          198    BIC:
1229.
Df Model:               1
Covariance Type:       nonrobust
=====

```

```

=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
Intercept      11.6375      0.674      17.263      0.000      10.308
12.967
facebook        1.2039      0.020      59.101      0.000        1.164
1.244
=====

```

```

=====
Omnibus:          20.420    Durbin-Watson:
1.948
Prob(Omnibus):    0.000    Jarque-Bera (JB):
23.417
Skew:             -0.785    Prob(JB):
8.22e-06
Kurtosis:         3.585    Cond. No.
61.7
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.